

# Anwendungsorientierte Softwareentwicklung

## Laborblatt 8: Es wird einiges vaporisiert

### Lernziele

- Vertiefung objektorientierter Programmierung
- Refactoring bestehender Programme
- Inkrementellen Softwareentwicklung: kleine Änderungen, testen und versionieren

### Schritt 1: Eigenes Repository klonen

Auch heute arbeiten wir wieder „nur“ mit Python. Ziel ist es, neue Features einzubauen. Zum Beispiel sollen Bälle, die man mit dem Paddle nicht trifft, am unteren Rand verschwinden. Und es soll – wie man es von Breakout erwartet – Steine (Bricks) geben, die beim Auftreffen des Balls verschwinden. Der Ball wird dabei reflektiert.

Obwohl unser Projekt aktuell nur aus wenigen Modulen besteht (`ball.py`, `breakout.py`, `controller.py`, `game.py`, `paddle.py`, `rocket.py`, `serial-reader.py`), darf man nicht unterschätzen, wie schwer es ist, bei einem mehrteiligen Programm den Überblick zu behalten – selbst wenn jede Datei nur wenige Zeilen lang ist.

Man braucht

- einerseits den **Blick fürs Grobe**: Was macht jedes Modul?
- andererseits den **Blick fürs Detail**: Wie macht das Modul, was es soll?

Zwischen diesen beiden Ebenen muss man flexibel hin- und herschalten. Beides gleichzeitig im Kopf zu behalten ist ab einem gewissen Komplexitätsgrad weder möglich noch nötig. Diese Fähigkeit wollen wir im ersten Schritt trainieren.

Wir gehen dabei so vor: Spiel starten, beobachten was es kann, dann den Code überfliegen, um die Struktur zu rekapitulieren.

### Vorbereitung: Zwei Terminals öffnen

Wie immer: Linkes Terminal zum Editieren, rechtes Terminal zum Ausführen.

### Rechtes Terminal (Spiel starten)

Wechselt mit `cd asp-uhl/python` ins Python-Verzeichnis und startet dann das Spiel mit `python breakout.py`.

Ihr solltet beobachten:

- Die Bälle werden sauber an allen Rändern (oben, unten, links, rechts) reflektiert. Genauer gesagt: der **Mittelpunkt** der Bälle bleibt immer innerhalb der Zeichenfläche – das wurde bei der Reflexion berücksichtigt.
- Mit dem **Paddle** kann man den Ball abprallen lassen. Aber Einfallswinkel = Ausfallswinkel gilt dabei **nicht**:
  - Trifft man den Ball in der Mitte, fliegt er senkrecht nach oben.
  - Ganz links → 30° nach links.
  - Ganz rechts → 30° nach rechts.
- Mit **Pfeiltaste Hoch** werden vom Paddle aus Raketen abgeschossen (kleine Kreisscheiben), die senkrecht nach oben fliegen und am oberen Rand verschwinden.

### Linkes Terminal (Code untersuchen)

Jetzt geht es darum, nachzuvollziehen, **wie** diese Features implementiert wurden. Es gibt dabei kein „richtig“ oder „falsch“ – das hängt vom Stil, Geschmack und auch vom Projektstand ab.

Ein möglicher Weg ist der Top-Down-Ansatz:

1. Starte mit der Hauptschleife in `breakout.py`.
2. Du wirst unter anderem sehen: Es gibt Klassen wie `Ball` und `Rocket`, von denen mehrere Instanzen erzeugt werden, die jeweils eine `draw`- und eine `move`-Methode besitzen.
3. Außerdem gibt es die Klasse `Paddle`, von der es nur eine Instanz gibt – mit (statischen) Methoden wie `left`, `right` und `draw`.
4. Danach kannst du gezielt die Dateien `ball.py`, `rocket.py` und `paddle.py` durchgehen.
5. Stelle dir beim Lesen immer wieder Fragen wie:
  - Warum habe ich das damals so gemacht?“
  - „Was genau bedeutet nochmal dieses oder jenes in Python?“
  - „Wie hängen die Module miteinander zusammen?“

Dabei fällt dir vielleicht auf: Manche Module wie `controller.py` oder `serial_reader.py` werden beim Start des Spiels **gar nicht verwendet**. Das ist kein Fehler – diese Dateien betreffen die **serielle Anbindung**, mit der man das Spiel später auch mit einem **Game-Controller** statt der Tastatur steuern kann.

Heute geht es aber **nur** um die Spiel-Features. Es reicht völlig, wenn du dir grob in Erinnerung rufst, *wofür* diese Module gedacht sind. Im Detail anschauen musst du sie dir heute nicht.

**One thing at a time.**

### Aufgabe

Geht diesen Weg selbständig durch. Startet das Spiel, beobachtet das Verhalten, untersucht den Code.

Wenn euch etwas unklar ist – fragt nach!

### Bemerkung

Beim Durchsehen des Codes ist uns eine Variante begegnet wie:

```
for r in rockets:
    r.draw()
    r.move()
rockets = [item for item in rockets if item.alive]
```

Das funktioniert in Python **scheinbar problemlos**, obwohl die Liste `rockets` **während der Iteration** verändert wird. Der Grund: In jeder Schleifenrunde wird eine **neue Liste** erzeugt und der Name `rockets` neu gebunden – aber die Schleife läuft **weiter über den ursprünglichen Iterator**, der noch auf die alte Liste zeigt. Es funktioniert also nur, weil Python **ineffizienten (und potenziell verwirrenden)** Code erlaubt.

```
for r in rockets:
    r.draw()
    r.move()
rockets = [item for item in rockets if item.alive]
```

Hier werden zunächst **alle Raketen bewegt und gezeichnet**, und **danach** werden alle entfernt, die nicht mehr sichtbar sind (`alive == False`).

Das ist **effizienter** und **leichter zu verstehen** – auch für das eigene Ich in ein paar Wochen 😊

## Schritt 2: Bälle, die man nicht trifft ...

Jetzt testen wir, ob das Rekapitulieren im ersten Schritt etwas gebracht hat.

**Ziel:** Wenn man mit dem Paddle einen Ball nicht trifft, soll dieser verschwinden, sobald er den unteren Rand der Zeichenfläche erreicht. Gemeint ist unter anderem: Das zugehörige Ball-Objekt soll aus der Liste `balls` entfernt werden.

### Aufgabe

Setzt dieses Feature um:

- Erkennt, wenn ein Ball „verloren“ geht (z. B. anhand seiner Y-Position).
- Entfernt das entsprechende Objekt aus der Liste `balls`.

**Tipp:** Überlegt, **wie das bei den Raketen gemacht wurde**. Auch dort verschwinden Objekte, wenn sie den Rand erreichen. Ziel ist es, diesen Mechanismus zu verstehen und **analog auf die Bälle zu übertragen**.

Wenn ihr **Fragen** habt, meldet euch. Wenn ihr meint, dass alles klappt, **zeigt uns**, was ihr verändert habt. **Wenn wir euch das OK geben**, dann ab damit ins Repository:

```
git commit -m "Ball verschwindet"  
git push
```

### Schritt 3: Space Invaders in Breakout?

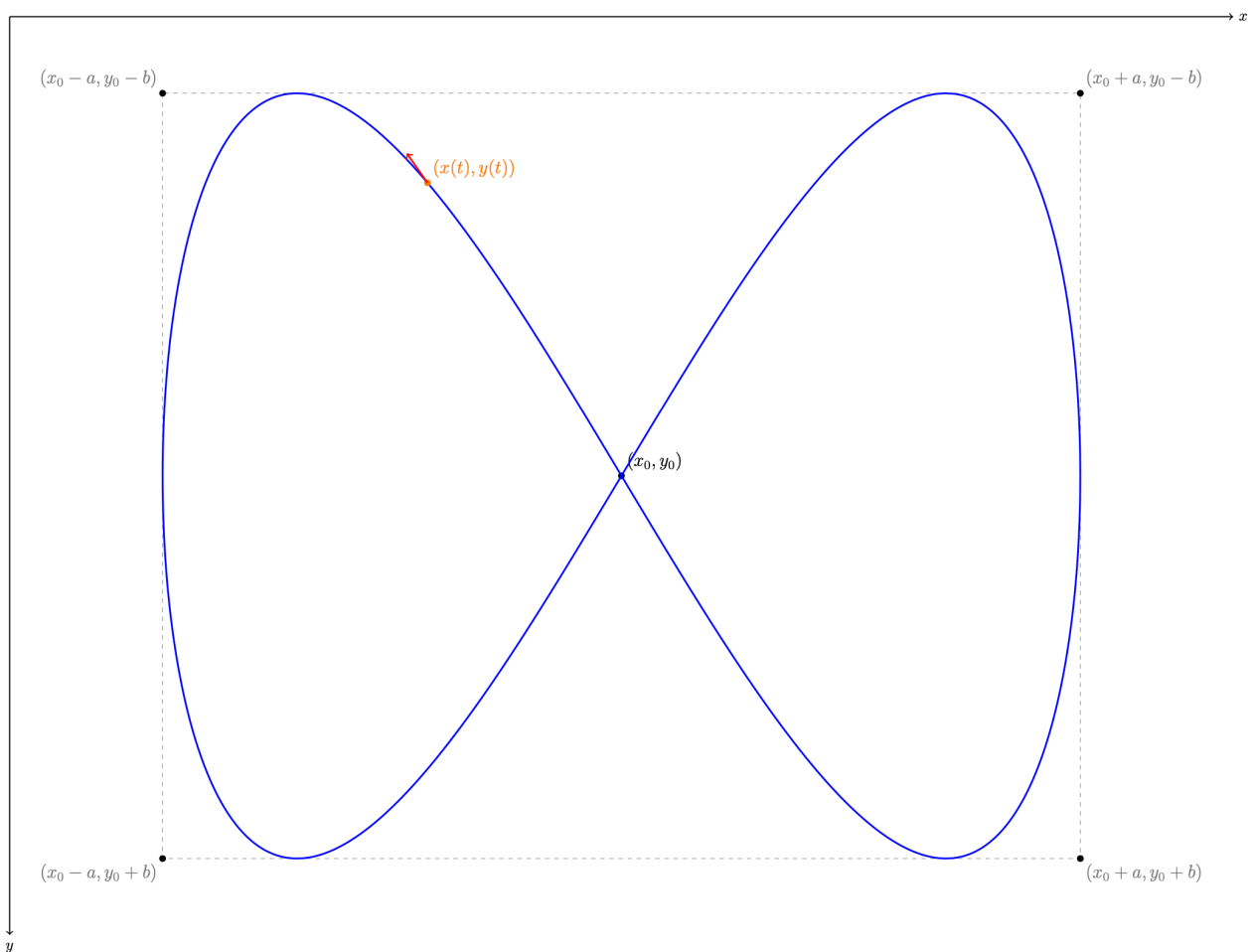
Warum eigentlich nicht? Das ist im wahrsten Sinne des Wortes eine Invasion – und gleichzeitig ein guter Moment, um zu merken, wie sehr es sich auszahlt, dass man sich den vorhandenen Code zuvor angeschaut hat.

**Idee:** Analog zur Klasse `Ball` kannst du jetzt eine Klasse `Ship` programmieren.

Jedes Objekt dieser Klasse stellt ein Raumschiff dar, das sich über die Zeichenfläche bewegt – ganz so wie die „Invaders“ im klassischen Spiel.

#### Bewegung entlang einer Kurve

Natürlich sollen sich die Raumschiffe **nicht einfach geradeaus bewegen** – das wäre zu leicht zu treffen. Stattdessen kannst du sie entlang einer interessanten Kurve fliegen lassen. Eine schöne Möglichkeit ist die sogenannte **Lemniskate von Gerono**. Die Spur dieser Kurve sieht aus wie eine **liegende Acht**:



Die Kurve ist zeitparametrisiert (mit  $t$  als Zeit) und lautet:

$$x(t) = x_0 + a \cdot \cos t, \quad y(t) = y_0 + b \cdot \sin 2t$$

Das bedeutet: Das Raumschiff bewegt sich innerhalb eines Rechtecks mit den Eckpunkten:

- **Links oben:**  $(x_0 - a, y_0 - b)$
- **Links unten:**  $(x_0 - a, y_0 + b)$
- **Rechts unten:**  $(x_0 + a, y_0 + b)$
- **Rechts oben:**  $(x_0 + a, y_0 - b)$

## Aufgabe: Raumschiffe ins Spiel bringen

Du sollst eine neue Klasse `Ship` schreiben – analog zur Klasse `Ball`. Ein Raumschiff bewegt sich entlang einer Lemniskate und kann (später) abgeschossen werden.

### Teil 1: Klasse Ship erstellen

Erstelle eine neue Datei `ship.py` und definiere dort eine Klasse `Ship`.

Da die Klasse der Klasse `Ball` sehr ähnlich ist, kannst du zunächst einfach eine Kopie machen:

```
cp ball.py ship.py
```

Dann änderst du die Datei schrittweise ab:

#### (A) Konstruktor

Ein Raumschiff soll so erzeugt werden können:

```
ship = Ship(0)
```

Der Parameter `0` steht für den Startwert der Zeit  $t = 0$ .

Folgende Attribute soll die Klasse besitzen:

```
self.x0 = Game.width / 2
self.y0 = Game.height / 3
self.a = 0.9 * Game.width / 2
self.b = 0.9 * Game.height / 3
```

Zusätzlich brauchst du:

```
self.t = t                # aktueller Wert der Zeit
self.alive = True         # False bedeutet abgeschossen
self.x, self.y = None, None # aktuelle Position (wird in move gesetzt)
self.move()               # move aufrufen um Position zu setzten
```

#### (B) Methode `move(self)`

Hier berechnest du die neue Position entlang der Lemniskate:

```
def move(self):
    self.x = self.x0 + self.a * math.cos(self.t)
    self.y = self.y0 + self.b * math.sin(2 * self.t) / 2
    self.t += 0.01
    while self.t > 2 * math.pi:
        self.t -= 2 * math.pi
```

Das Zurücksetzen von  $t$  ins Intervall  $[0, 2\pi)$  ist **nicht zwingend notwendig** – Python erlaubt es problemlos, dass Zahlen beliebig groß werden (und keine Sorge: in diesem Fall wird der Speicher zu euren Lebzeiten nicht voll 😊). Aber: Es ist eleganter und speicherschonender – und

ein echtes Ingenieursherz schlägt bei so etwas höher ✅

#### (C) Methode `draw(self)`

Zeichne das Raumschiff einfach als Kreisscheibe mit größerem Radius als der Ball – und gerne mit einer anderen Farbe.

### Teil 2: Raumschiffe ins Spiel einbauen

Nun müssen die Raumschiffe in `breakout.py` eingebunden werden – analog zu den Bällen:

1. Klasse Importieren: `from ship import Ship`
2. Eine Liste von Schiffen anlegen, z.B. `ships = [Ship(0), Ship(0.5), Ship(1)]`
3. Bewegen, zeichnen und abgeschossene Schiffe aus der Liste entfernen

Wenn euer Code läuft:

```
git add ship.py
git commit -a -m "Raumschiffe!"
git push
```

## Schritt 4: Jetzt wird's ernst!

Natürlich sollte man mit einem **Ball** oder einer **Rakete** ein **Raumschiff abschießen** können. Das funktioniert, indem man prüft, ob der Abstand zwischen einem Raumschiff und einem Ball oder einer Rakete **klein genug** ist.

Hier ist ein Beispiel, wie man prüft, ob ein **Ball** einem **Raumschiff** zu nahe gekommen ist:

```
for s in ships:
    for b in balls:
        dist = math.hypot(b.x - s.x, b.y - s.y)
        if dist <= 20:
            s.alive = False
```

### Was passiert hier genau?

Wir wollen herausfinden, ob ein **Ball ein Raumschiff getroffen hat**. Dazu müssen wir für **jedes Raumschiff** den Abstand zu **jedem Ball** berechnen. Dafür werden zwei Schleifen **ineinander geschachtelt**:

- Die **äußere Schleife** läuft mit der Variablen **s** über alle Raumschiffe in der Liste **ships**.
- Für **jeden einzelnen Wert von s** durchläuft die **innere Schleife** mit der Variablen **b** alle Bälle in der Liste **balls**.

In der **innersten Schleife** liegen also immer genau **ein Raumschiff s** und ein Ball **b** vor – für jedes mögliche Paar aus Raumschiff und Ball.

Dann wird der Abstand zwischen diesen beiden Punkten mit dem guten alten Pythagoras berechnet:

$$\text{dist} = \sqrt{(b.x - s.x)^2 + (b.y - s.y)^2}$$

Wenn der Abstand kleiner oder gleich 20 ist, dann wurde das Raumschiff getroffen. Wieso 20? Bei mir hat ein Ball den Radius 5 und ein Raumschiff den Radius 15.

### Aufgabe:

1. **Importiert** in `breakout.py` das Modul `math`, falls noch nicht geschehen:

```
import math
```

**Fügt den obigen Code** an geeigneter Stelle in die Hauptschleife ein:

- **nachdem** alle Objekte bewegt und gezeichnet wurden
- **bevor** die Listen `balls` und `ships` bereinigt werden (also die „Leichen“ entfernt werden)

**Testet**, ob man mit einem Ball jetzt ein Raumschiff abschießen kann.

2. Eigentlich war der Sinn der Raketen, dass man damit etwas abschießen kann. Ändert den Code so ab, dass in der inneren Schleife die Liste `rockets` statt `balls` durchlaufen wird. **Testet**, ob man jetzt mit einer Rakete ein Raumschiff abschießen kann.
3. Noch besser wäre es natürlich, wenn man **sowohl mit einem Ball als auch mit einer Rakete** ein Raumschiff abschießen könnte – ohne doppelten Code. Naheliegender wäre z. B. folgender Ansatz:

```
for b in [balls, rockets]:
    ...
```

Man könnte meinen, dass damit **alle Bälle und alle Raketen** durchlaufen werden. **Aber überzeugt euch selbst, dass das nicht klappt!**

### Denn was passiert hier wirklich?

`[balls, rockets]` ist eine Liste, die genau zwei Elemente enthält: nämlich die Liste `balls` und die Liste `rockets`. Das heißt: Die Schleife läuft zweimal – einmal mit `b = balls`, einmal mit `b = rockets` – und `b` ist dann jeweils eine ganze Liste, nicht ein einzelnes Objekt.

4. Mit `balls + rockets` erzeugt man eine neue Liste, die alle Elemente aus `balls` und aus `rockets` enthält.

Wenn man also schreibt:

```
for s in ships:
    for b in balls + rockets:
        dist = math.hypot(b.x - s.x, b.y - s.y)
        if dist <= 20:
            s.alive = False
```

dann sollte alles wie gewünscht funktionieren: Man kann sowohl mit einem Ball als auch mit einer Rakete ein Raumschiff vaporisieren 💥

Testet, ob das klappt. Wenn ja – dann wisst ihr, dass sich die Mühe gelohnt hat. Und dann, wie immer ins Repository damit:

```
git commit -a -m "shoot to kill"
git push
```

## Schritt 5: Steine!

Jetzt kommt eine **Wand aus Steinen (Bricks)** ins Spiel – was natürlich Sinn macht, wenn sich das Spiel **Breakout** nennt 😊

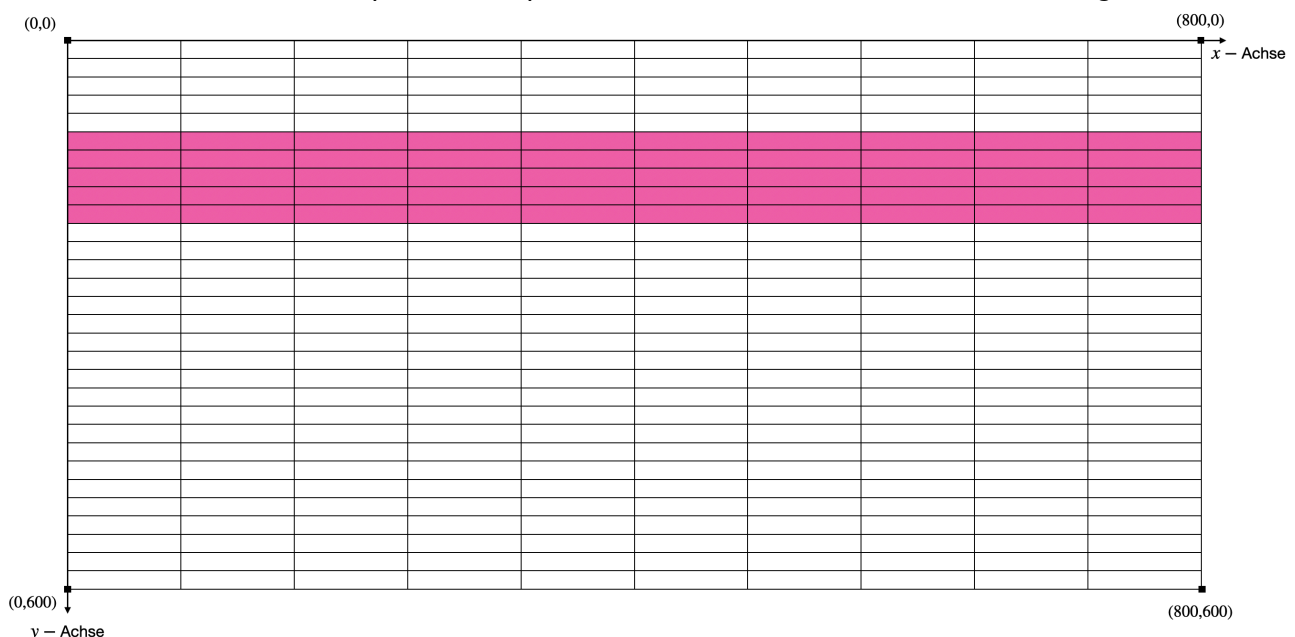
In einer **ersten Version** sollen die Bälle einfach noch **durch die Bricks hindurchfliegen**. Es geht zunächst **nicht** um die Kollision, sondern darum:

🧩 **Wo befindet sich auf der Zeichenfläche ein Brick – und wo nicht?**

### Das Raster

Der Einfachheit halber soll bei uns ein **Brick 80 Pixel breit und 20 Pixel hoch** sein. Da die Zeichenfläche **800 Pixel breit und 600 Pixel hoch** ist, ergibt sich daraus ein **Raster mit 10 Spalten und 30 Zeilen**. Natürlich ist nicht jede Zelle in diesem Raster mit einem Brick belegt.

Zunächst könnten zum Beispiel nur die paar der Zeilen Bricks enthalten – wie in folgender Skizze:



### Ein bisschen Mathematik

Ein **Raster** mit **10 Spalten** und **30 Zeilen**, in dem in jeder Zelle entweder ein Brick ist oder nicht, kann man sich als **Matrix** vorstellen.

Jeder Eintrag ist dann entweder: **1** → Brick ist vorhanden, **0** → keine Brick vorhanden. Und weil wir als Mathematiker Wert auf Präzision und Konventionen legen: Wir sprechen von einer **30×10-Matrix** (Zeilen × Spalten).

### Umsetzung in Python: `numpy`

In Python kann man mit dem Modul `numpy` sehr effizient mit Matrizen arbeiten. Für unsere Zwecke brauchen wir nur:

- eine **Null-initialisierte 30×10-Matrix**.
- Die **Indizierung** beginnt **bei 0**: Zeilenindex geht von 0 bis 29, der Spaltenindex von 0 bis 9.
- Zugriff auf einzelne Einträge zum **Lesen und Schreiben**.

### Die Klasse `Wall`

Erstellt eine neue Datei `wall.py` und schreibt dort folgenden Code:

```

1 import pygame
2 import numpy
3 from game import Game
4
5 class Wall:
6     brick = numpy.zeros((30, 10))
7
8     def init():
9         for i in range(4, 8):
10             for j in range(0, 10):
11                 Wall.brick[i, j] = 1
12
13     def draw():
14         for i in range(0, Wall.brick.shape[0]):
15             for j in range(0, Wall.brick.shape[1]):
16                 if Wall.brick[i, j] == 0:
17                     continue
18                 w = 80
19                 h = 20
20                 x = j * w
21                 y = i * h
22                 r = (x + 1, y + 1, w - 1, h - 1)
23                 pygame.draw.rect(Game.screen, (223, 105, 165), r)

```

### Erklärung für `wall.py`

- In **Zeile 2** wird das Modul `numpy` importiert.
- Die Klasse hat ein **Klassenattribut** `brick`, das eine **30×10-Matrix voller Nullen** ist.
- Die Methode `init` setzt in den **Zeilen 4 bis 7** alle Matrixeinträge auf **1** – dort sind Bricks!
- In der Methode `draw` wird die Wand gezeichnet:
  - Mit `Wall.brick.shape[0]` und `Wall.brick.shape[1]` fragt man jeweils die Zeilen- und Spaltenzahl ab.
  - Zwei verschachtelte Schleifen durchlaufen die Matrix.
  - `i` steht für den **Zeilenindex**, `j` für den **Spaltenindex**.
  - Wenn der Eintrag `0` ist, wird mit `continue` zum nächsten Schleifendurchlauf gesprungen.
  - Sonst wird ein Rechteck gezeichnet:
    - Breite `w = 80`, Höhe `h = 20`
    - obere linke Ecke bei `x = j * w`, `y = i * h`
    - leicht verschoben (+1) und verkleinert (-2) für sichtbare Lücken



**Aufgaben:**

1. **Baut die Brick-Wand ins Spiel ein**, indem ihr die Klasse Wall in `breakout.py` einbindet:
  - (A) Importiert die Klasse am Anfang der Datei: `from wall import Wall`
  - (B) Ruft nach `Game.init()` auch die Initialisierung der Wand auf: `Wall.init()`
  - (C) Ruft innerhalb der Hauptschleife auch `Wall.draw()` auf – am besten **nachdem die Bälle gezeichnet wurden**: `Wall.draw()`
2. **Testet**, ob die Brick-Wand wie erwartet angezeigt wird.  
Wenn alles klappt – rein ins Repository:
 

```
git add wall.py
git commit -a -m "Die Wand steht"
git push
```

**Schritt 6: Jetzt räumen wir die Steine aus dem Weg!**

Die Bricks sind aktuell reine Dekoration. In diesem Schritt sollen sie ihrer Bestimmung gerecht werden: **Sie sollen von den Bällen durchschlagen werden.**

**Zunächst einmal ganz simpel:**

Die Bälle prallen noch nicht ab, sondern fliegen ungebremst weiter – **der Brick verschwindet, der Ball nicht.**

Natürlich könnte man ganz genau berechnen, **ob und wo ein Ball gegen einen Brick prallt**: Für jeden Frame kennen wir ja die alte und die neue Position eines Balles. Damit ließe sich berechnen, welche Kante eines Bricks durchquert wird.

💡 Wer Lust auf eine kleine Mathe-Knochelei hat, darf das gerne selbst „aus-x-en“.

Wir machen es uns an dieser Stelle einfacher – mit einem **näherungsweisen Verfahren**:

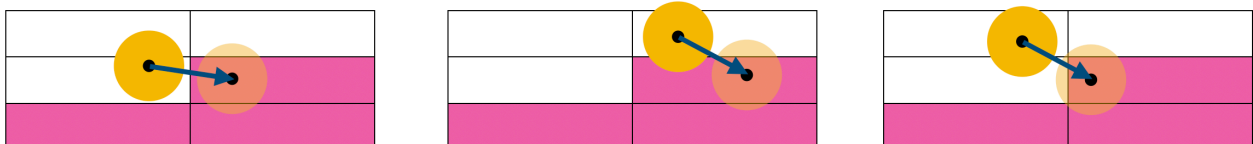
- Aus der **alten Position** `x1, y1` des Balls berechnen wir, **in welcher Rasterzelle** sich der Ball vorher befand: das ergibt ein Paar `i1, j1` (Zeile, Spalte).
- Aus der **neuen Position** `x1, y1` berechnen wir entsprechend, in welche **Rasterzelle** der Ball jetzt gelangt ist: das ergibt ein Paar `i2, j2` (Zeile, Spalte).

Wenn sich der Ball **von einer Zelle mit Brick in eine andere Zelle mit Brick bewegt**, dann gehen wir davon aus, dass dabei ein Brick durchschlagen wurde – und **setzen den Eintrag in der Matrix** `Wall.brick[i2, j2]` auf 0.

**Und mehr noch:** Wir können sogar näherungsweise abschätzen, **wie** der Brick getroffen wurde:

- Ist `i1 ≠ i2`, dann wurde vermutlich die **obere oder untere Kante** durchschlagen.
- Ist `j1 ≠ j2`, dann wurde vermutlich die **linke oder rechte Kante** durchschlagen.
- Wenn sowohl `i1 ≠ i2` und `j1 ≠ j2` gilt, dann müsste sogar eine Ecke getroffen worden sein, was in der Praxis aber eher selten der Fall ist. An solchen Fällen erkennt man auch gut die Grenzen dieser Näherung.

**Die folgende Skizze zeigt drei typische Fälle** – und hilft, ein Gefühl dafür zu bekommen, wie gut (oder wie schlecht) diese Methode in der Praxis funktioniert:



## Wie berechnet man aus den (x,y)-Koordinaten die Spalte und Zeile im Raster?

- Wenn man die **x-Koordinate** durch die **Breite** einer Zelle teilt und das Ergebnis auf eine ganze Zahl abrundet, erhält man die **Spalte**. Das geht in Python bequem mit Integer-Division (`//`).
- Dasselbe gilt für die **y-Koordinate**: Teilt man sie durch die **Höhe** einer Zelle und rundet ab, erhält man die **Zeile**.

Ergänzt die Klasse Wall um folgende Klassenmethode:

```

13     def collision(x2, y2):
14         j2 = int(x2 // 80)
15         i2 = int(y2 // 20)
16         if i2 >= 0 and i2 < 30 and j2 >= 0 and j2 < 10:
17             if Wall.brick[i2, j2]:
18                 Wall.brick[i2, j2] = 0

```

### Was macht diese Methode?

- In dieser Version bekommt `collision` sowohl zunächst nur die **neue Position** (`x2, y2`) des Balls übergeben.
- In **Zeile 14 und 15** wird aus `x2` und `y2` jeweils die Spalte `j2` und die Zeile `i2` berechnet – mit Integer-Division durch die Zellbreite bzw. -höhe.
- In **Zeile 16** wird geprüft, ob `i2` und `j2` überhaupt gültige Indizes im Raster sind – denn es kann vorkommen, dass sich der Ball schon **außerhalb** der Zeichenfläche befindet (z. B. vor der Korrektur an den Wänden).
- In **Zeile 17** wird geschaut, ob sich **in der getroffenen Zelle ein Brick befindet**.
- In **Zeile 18** wird dieser dann entfernt – wir „vaporisieren“ den Brick also durch Setzen des Matrixeintrags auf `0`.

### Aufgaben:

1. In der `move`-Methode der Klasse `Ball` soll jetzt überprüft werden, ob der Ball einen Brick trifft. Dazu sind nur ein paar Änderungen in `ball.py` nötig:
  - (A) Die Klasse `Wall` muss importiert werden: `from wall import Wall`
  - (B) Ruft in der `move`-Methode (nachdem `new_x` und `new_y` berechnet und korrigiert wurden) die Kollisionserkennung auf: `Wall.collision(new_x, new_y)`

**Testet, ob Bricks nun beim Kontakt mit einem Ball verschwinden.** Der Ball sollte dabei durch die Bricks hindurchfliegen wie durch Butter – denn aktuell bleibt sein Kurs völlig unbeeinflusst. Es wird lediglich geprüft, ob er dabei einen Brick trifft – und dann entfernt.

2. **Jetzt kommt für heute das große Finale.**

Der Ball soll beim Kontakt mit einem Brick nicht nur den Brick verschwinden lassen, sondern auch abprallen. Dazu ändert in der `move`-Methode der Klasse `Ball` den Aufruf ab auf:

```

self.vx, self.vy = Wall.collision(self.x, self.y, new_x, new_y,
                                self.vx, self.vy)

```

Übergeben wird jetzt die alte Position (`self.x, self.y`), die neue Position (`new_x, new_y`) sowie der aktuelle Geschwindigkeitsvektor (`self.vx, self.vy`).

Die Methode soll nun einen (ggf. angepassten) Geschwindigkeitsvektor zurückgeben. Jetzt seid ihr gefragt, `Wall.collision(...)` entsprechend zu erweitern:

1. Wird die **obere oder untere Kante** eines Bricks durchschlagen, soll die **y-Komponente** des Geschwindigkeitsvektors „umgedreht“ werden.
2. Wird die **linke oder rechte Kante** eines Bricks durchschlagen, soll die **x-Komponente** des Geschwindigkeitsvektors „umgedreht“ werden.
3. Wenn sich **beide Indizes** ändern, prallt der Ball also an einer Ecke ab – und **beide Komponenten** sollen negiert werden.

Testet, ob der Ball nun korrekt abprallt und gleichzeitig den Brick entfernt. Wenn alles klappt:

```

git commit -a -m "Bricks können jetzt was ab"
git push

```